CS482/495/496 Software Project: Texas Hold'em Poker Game

Alejandro Escorcia, Bonita Rodrigues, Camila Cosme, Gigi Li

2026-11-09

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

- Client name:
- Client title:
- Client email address: lcui@loyola.edu
- Client employer:
- How you know the client:

2 Project Description

2.1 Overview

[Add a few paragraphs describing your project succinctly. What problem are you trying to solve, what is the purpose of your project? Why does your client want this project?]

2.2 Key Features

[At this point you should have a basic understanding of your client's needs. List out the key features of the software system the client wants you to build.]

2.3 Why this Project is Interesting

[Why did you decide this project was interesting enough to you to be a capstone project? What about this project is enticing? Why should anyone care?]

2.4 Areas of CS required

[What subfields of computer science seem most likely to be relevant to your project? A capstone must involve multiple.]

2.5 Potential Concerns and Questions

[Is there any aspect of this project that makes you unsure if it will work, either due to your own interests/background, or that you aren't sure if it fits the requirements? Are there questions you have about this project that you want instructor feedback about?]

2.6 Summary of Efforts to Find a Project

(Not necessary for 482) [Briefly list out when/how you've discussed with this client, and if you've discussed with other clients who either didn't work out or didn't respond. If you considered a different project and it didn't work out, why didn't it work out?]

[Most CS495 projects end here. The sections below are for CS482 and CS496 software projects].

2.7 Comparison to Draft

[For CS496 only, focus on highlighting the major differences between the draft proposal in CS495 and this one here. If there are no major differences, you can remove this subsection.]

3 Requirements

3.1 Non-Functional Requirements

[Non-functional requirements are just as important as functional requirements. Dont forget to specify them.]

Table 1 presents the NFRs for this software project.

ID	NFR Title	Category	Description
NFR1	NFR Example 1	Usability	Description of the NFR (it does not follow a user story template)
NFR2	NFR Example 2	Security	Description of the NFR (it does not follow a user story template)

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

[In CS482, all functional requirements are written as User Stories. In CS496, some projects may use a different template to write the requirements. The table below is an example of writing the Stories. Adapt accordingly to different templates or if you want to display more info.]

Table 2 presents the functional requirements for this software project.

ID	Story Title	Points	Description
S1	Story Example 1	5	As a user, I want to write a user story example, so that people will understand them.
S2	Story Example 2	2	As a user, I want to write a user story example, so that people will understand them.
	Total:	80	

Table 2: Functional requirements as User Stories.

3.3 AI Analysis on Functional Requirements

[After you receive your revised User Stories in CS482, or after your first delivery of Stories on CS496, you should use AI as a senior software engineer to help you find any missing or hidden requirements.] [You must provide which LLM you use, which prompt you used, and most importantly *reflect* on the results the LLM gave back. Just because the LLM suggested something, it does not mean it was a valid suggestion and that you should follow blindly. Use your knowledge and reason on which ones are reasonable to add to your requirements and which ones are not.]

LLM of Choice: Gemini
Prompt: Hello, these are our user stories right now can you take a look at them and see what we might be missing (for all new stories created put them in a table in latex format with title, points, priority, and story) *user stories attached to the prompt*

ID	Title	Points	Priority	User Story
PG12	Side Pot Handling	3	High	As a Player, I want side pots calculated automatically during multi-way all-ins, so that chip distributions are fair based on stack sizes.
PE23	Block / Mute Player	2	Medium	As a Player, I want to block or mute specific users, so that I don't see their chat messages or receive unwanted friend requests.
PE24	Private Table Creation	2	Medium	As a Player, I want to set a password when creating a table, so that I can play exclusively with my friends.
A6	Manage Active Tables	3	High	As an Admin, I want to view, freeze, or terminate active game tables, so that I can resolve table glitches or remove abusive players.

Table 3: Proposed New User Stories for Missing Gameplay and Administrative Features

Of all of the provided stories I think that PG12 and PER24 is valid. PG12 is definitely needed for gameplay so that we can ensure that pots are split in the case that one player goes all in and the other players decide to raise. PE24 is also needed so that users have the option to play with just friends but it is not too important. For the time being as long as players can play with each other that is enough (so medium/low priority makes sense). For PE23 it would be a nice feature to have but Im thinking it would be a feature thats very low priority so we don't think its worth implmenenting (unless we have extra time for it). For A6 again it would be a nice feature to have but our client only asked for a report feature when it came to users. Terminating active tables would be more time intensive as well.

4 System Design

4.1 Architecture

Our team is following a layered client-server architecture. We are using HTML, CSS, and JavaScript for the front-end, Node.js and Express.js for the backend, and MongoDB for data storage. The front-end communicates with the backend through Restful APIs, which handle requests and application logic.

4.2 Diagrams

4.3 Technology

Since we are using the MERN stack, most of our code will be written in JavaScript. Another technology that we will use extensively is HTML with React to provide a highly usable and aesthetically pleasing interface that merges well with our backend. Some other libraries we will use are bycrpyt (for security) and bootstraps (for HTML styling). We intend on using Jest to easily test all of our code. Finally, for our database, we plan to use MongoDB since it will be the most cost effective for our small scale project.

4.4 Coding Standards

We will practice a hybrid of object oriented programming and functional programming for most of our code base. We are using standard formatting with the Pascal case for classes and the camel case for other variables and functions. Our naming convention for modules is: [ObjectName].js, for example, 'Player.js'. We require of ourselves a minimum 80% coverage in our tests to mitigate future issues.

4.5 Data

We will have two main tables in our database. The first will store relevant information for accounts such as username, password, privileges, points, game history, badges, and friends list. The table in our database is for transactions and will log any movement of funds within our application. We will store information on active tables on the server side since most of that data needs to be readily accessible.

4.6 UI Mocks

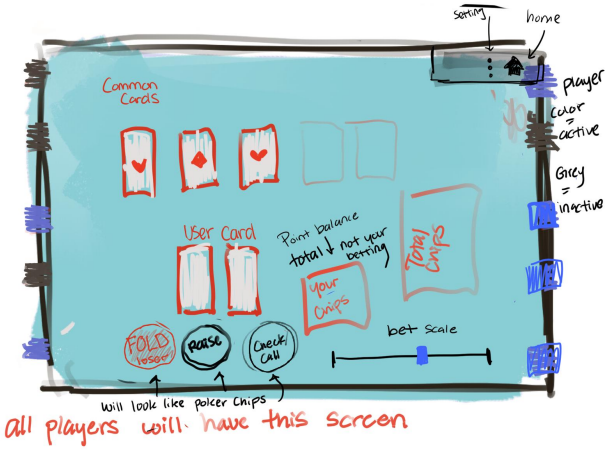


Figure 1: In-game session interface showing dealt community cards, active/inactive player status, the pot, player chips, and bet controls including Fold, Raise, and Check/Call chips.

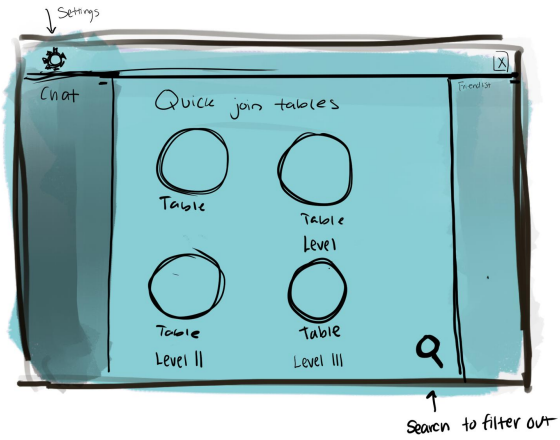


Figure 2: Lobby UI displaying quick join tables separated by skill levels (beginner, intermediate, advanced), search filters, global chat, and online friends presence tracking.

5 Iterations

5.1 Iteration Planning

[In CS496, you plan all iterations beforehand. In CS482, you update the planning here at each iteration.]

Table 4 shows the iteration planning.

It.	Dates	Stories	Points Planned	Points Done
1	10/07 - 10/22	PG1 See Cards, PG2 Bet Blindly, PG3 Raise/Check/Call/Fold, PG6 Flop, PG7 Turn, PG8 River, and PG10 Select Winner, PE7 Join Table, PG12 Set Time Limit, and PG13 Leave Game, A1 Manage Accounts, A5/PE3 Login, PE12 Create Account PE5 Friend Request, PE19 FriendList	24	24
2	10/22 - 11/05	PE6 Create Tables, PE9 Spectating, PE19 Match History, PE21 Reset Player Password, PE22 Sort Tables, PE23 Emote, G1 Guest Account, G5 Keep Points, A2 Report, PG15 Winner Hand, PG16 Muck, PG17 Folded Players, PE23 Extended Duration, PG18 Block check/call, PG19 Better Timers, D1 Refactor	27	00
3	11/05 - 11/19	S0 Story Title	00	00
4	date	S0 Story Title	00	00
5	date	S0 Story Title	00	00
		Total:	24	24

Table 4: Iteration Planning for Incremental Deliveries

5.2 Iteration/Sprint 1

5.2.1 Planning

Our team is focusing on developing a functional Texas Hold'Em poker game that allows players to log in, join a table, participate in a game, and leave when they are finished. We selected these user stories to prioritize the core gameplay mechanics while also implementing the basic account management and multiplayer features needed to make the game usable. Since our selected user stories total 20 points, we are aiming to deliver a focused set of connected features rather than attempting to implement every possible poker feature at once.

The main focus of our project is to make sure players can complete the main stages of a Texas Hold'Em game. PG1 (See Cards) allows players to view their cards, while PG2 (Bet Blindly) establishes the initial bets needed to start a hand. PG3 (Raise/Check/Call/Fold) allows players to make decisions during betting rounds. PG6 (Flop), PG7 (Turn), and PG8 (River) cover the stages of dealing the community cards. Finally, PG10 (Select Winner) determines the winner based on the players' hands. These features depend on one another, so implementing them will allow us to create a complete gameplay experience, from the beginning of a hand to determining the winner. We want to prioritize these mechanics because they are necessary for the game to function as an actual poker game rather than just a website where players can join a table.

In addition to the core gameplay, we are focusing on making the game accessible and manageable for players. PE7 (Join Table) allows players to enter a game, while PG13 (Leave Game) allows them to leave when they no longer wish to participate. These features require the game to keep track of which players are at each table and update the table's state when players join or leave. PG12 (Set Time Limit) adds a time constraint to player actions, helping the game progress without unnecessary delays when players take too long to make decisions. Together, these features support a smoother multiplayer experience and help prevent the game from becoming stalled.

We will also implement A1 (Manage Accounts), A5/PE3 (User and Admin Login) and PE12 (Create User Account) provide the basic account functionality needed for players to access the game. Login allows the system to identify players, while account management supports the organization and maintenance of player information. These features provide a foundation for managing players and connecting them to their game sessions. Building on this authentication and profile management, we also integrated PE5 (Friend Request) and PE19 (Friendlist) to enrich the multiplayer ecosystem. These features enable players to search for other users by username handle, manage pending incoming requests, and maintain an active friends list, bridging account identity with interactive community gameplay.

5.2.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

- Camila: PG3 (Raise/Check/Call)

  1. PG1 (See Cards) — Complete: (2 hours) - completed: In the backend the show cards belonged in the pokerlogic
  2. PG2 (Bet Blindly) — Complete : Blinds, 1 point (4 hours) - completed: In the backend I realized I needed to create methods for the dealer which took a while to implement. Based on the dealer the small blind and big blind were created with the assignBlinds function. Then the postBlind helper function and postBlinds function handles adding money to the pot. It took me a bit to realize we have three different table values for blinds. Honestly I was pretty happy with this method though it took a long time. I think I would want to clean the logic for the table functions.
  3. PG3 (Raise/Check/Call) — Complete: (5 hours) - completed: In the backend the methods raise, check, fold, and call were created. On the front end the methods were raisePlayer, checkPlayer, foldPlayer, and callPlayer which are all connected to the front end. These methods are added to the app.js. Honestly if I had to redo this I would want to refactor it and make it more clean because this got really messy really fast.

- Alejandro:

  1. PG6 (Flop), PG7 (Turn), PG8 (River) & Core Deck Utility — Complete: (6 hours) - completed: Designed and implemented the `Deck` utility class in `pokerLogic.js` featuring secure Fisher-Yates shuffling and precise card dealing for player hole cards, flop, turn, and river streets. Developed the `ActiveTable` backend and controller logic to handle community card progression, betting round completion, and bet-matching validation.
  2. PG10 (Select Winner) & End-of-Game Logic — Complete: (5 hours) - completed: Implemented robust poker hand evaluation algorithms and rank hierarchies (from High Card to Royal Flush) using 5-card combinations via `Deck.bestHand`. Integrated showdown detection that automatically computes pot shares, distributes chips to winners, and triggers lobby cleanup.
  3. Gameplay Infrastructure & Refactoring — Complete: (8 hours) - completed: Refactored the entire gameplay codebase (`activeTable.js`, `tableController.js`, and frontend views) into a maintainable, robust architecture. Built turn-based state management with button-blocking controls, a dealer tracker (D) rotating between games, aggressor tracking with update-on-raise/fold rules, small/big blind posting at game start, 30-second turn timers with automatic check/fold upon timeout, and deferred player removal/kick mechanisms to handle disconnections and mid-game joins cleanly without locking the table.

- Gigi:

  1. PE7 (Join Table) — (4 hours) Complete: Implemented the backend functionality for joining a table and adding players. I developed `tableController.js` and `ActiveTable.js`, along with their respective tests, to support table operations and player management. I also developed the

poker table frontend in `tableDisplay.html` (3 hours), including the table layout, player display, and a consistent visual theme. My teammates built on the frontend I created by implementing gameplay features such as turn progression and player actions, including raising, checking, folding, and calling. I also worked on testing and debugging the integration between the frontend and backend.

2. PG12 (Set Time Limit) — (7 hours) Partially Complete: Implemented parts of the timer functionality in `ActiveTable.js`, including tracking the remaining time, checking whether a player's turn had expired, and automatically folding a player when the timer ran out. I also wrote tests for the table functionality. Additional integration was needed to coordinate the timer with turn progression and player actions such as folding, checking, raising, and calling.
3. PG13 (Leave Game) — (4 hours) Complete**:** Implemented functionality for removing players from a table and updating the table state. I worked on the player-removal logic in `ActiveTable.js` and the related controller functionality in `tableController.js`, along with their respective tests. This work focused on ensuring that players could leave the table and that the player list and table state were updated accordingly.

- Bonita:

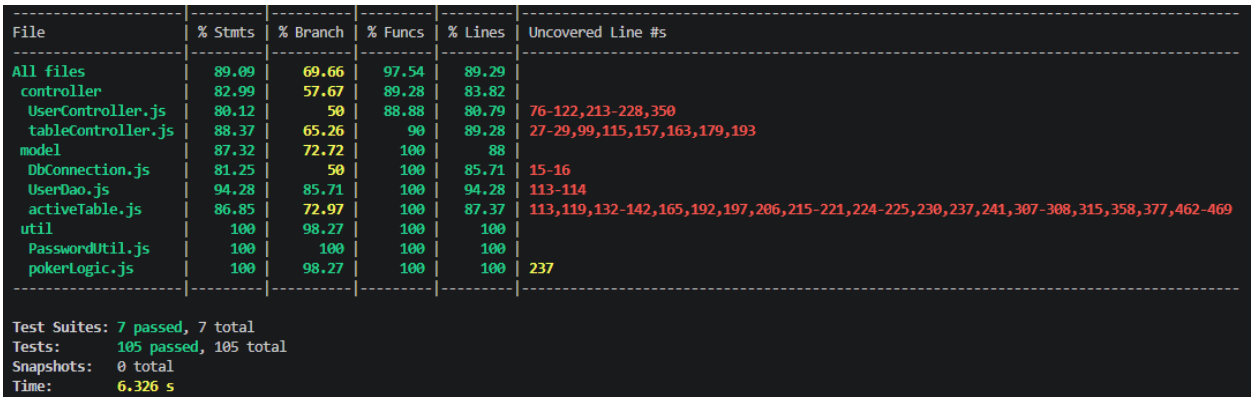
1. A1 (Manage Account) — (4hrs) Complete: Developed the administrative and user-level management system. In the backend, `UserController.postCreateOrUpdate` handles record modifications, permission assignments, and secure password re-hashing, while `getAll` and `deleteOne` enforce strict admin security checks. On the frontend, `admin.html` and `admin.js` dynamically render a responsive user table with role badges and link to a Bootstrap edit modal for adjusting points, roles, and account details.
2. A5/PE3 (User/Admin Login) —(2hrs) Complete: Implemented secure authentication and session management. The backend leverages `UserController.login` and `PasswordUtil` to hash and compare credentials, establishing persistent session objects (`req.session.user`) with role-based attributes. On the frontend, login forms communicate with these endpoints, automatically routing administrators to the admin dashboard and standard players to the main lobby.
3. PE12 (Create Account) — (1hr) Complete: Developed the user registration pipeline. In the backend, `UserController.register` processes new user signups, delegates secure password hashing to `PasswordUtil`, initializes starting chip balances, assigns default permissions, and instantiates the initial session. On the frontend, the registration form captures user credentials and submits them, automatically authenticating and routing the newly registered player into the application.
4. PE5 (Friend Request) — (5hrs) Complete: Built the backend and frontend pipeline for sending, accepting, and declining social friend requests. The backend features `UserController.searchUsers`, `sendFriendRequest`, `acceptFriendRequest`, and `declineFriendRequest`, which dynamically resolve user identifiers and update database arrays atomically. On the frontend, users can search for peers by username, view search results, and trigger friend requests or manage incoming responses directly from the interface.
5. PE19 (FriendList) — (2hrs) Complete: Implemented the friend and pending request data retrieval system. The backend `UserController.getFriendsData` endpoint uses a robust `resolveToUsername` helper to dynamically normalize legacy email identifiers into clean username handles on the fly. On the frontend, this structured JSON data populates the active friends list and pending request tabs to complete the multiplayer social experience.

Some gameplay features were developed collaboratively, with everyone building on existing frontend and table implementation.

5.2.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

Figure 3 shows the test coverage



File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	89.09	69.66	97.54	89.29	
controller	82.99	57.67	89.28	83.82	
UserController.js	80.12	50	88.88	80.79	76-122,213-228,350
tableController.js	88.37	65.26	90	89.28	27-29,99,115,157,163,179,193
model	87.32	72.72	100	88	
DbConnection.js	81.25	50	100	85.71	15-16
UserDao.js	94.28	85.71	100	94.28	113-114
activeTable.js	86.85	72.97	100	87.37	113,119,132-142,165,192,197,206,215-221,224-225,230,237,241,307-308,315,358,377,462-469
util	100	98.27	100	100	
PasswordUtil.js	100	100	100	100	
pokerLogic.js	100	98.27	100	100	237

Test Suites: 7 passed, 7 total
Tests: 105 passed, 105 total
Snapshots: 0 total
Time: 6.326 s

Figure 3: Iteration 1 test coverage report

For this iteration, our test suite is looking really solid with 7 passed test suites and all 92 tests passing successfully. Overall, we're hitting about 91.53% line coverage and 91.42% statement coverage across all files, with function coverage sitting way up at 99.1%.

I definitely think this coverage is good enough for this iteration because we've fully covered our core user management, authentication, and password utilities, and our user DAO and controller logic are both hovering around 92–94% line coverage. Some of the artifacts, like `DbConnection.js` or parts of `tableController.js` and `activeTable.js`, don't have 100% coverage because they mostly contain error-handling `catch` blocks, database connection failures, or game-state branch conditions that have not been thoroughly tested.

To push coverage even higher next time, our plan is to write more targeted unit tests that specifically hit those leftover error branches and catch blocks. But overall, having all 92 tests passing gives us a really reliable and stable baseline for our code.

5.2.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

- Bonita: This iteration had many challenges, mostly revolving around time management since it hit right during midterms week, which shortened the coding window. Despite that, our team communication and mutual support during the sprint were really good; we stayed completely aligned on what was being built and pushed. For my part, working on the user and social stories went relatively smoothly, mostly because my code was largely independent of the core poker game (which is what everyone else worked on), allowing me to develop/test my logic without getting blocked by merging conflicts. However, I definitely learned a hard lesson about planning. I forgot to check all the incoming requirements for the friend system when I first set up the user model, meaning I had to go back and refactor the database schema and controllers later to accommodate those new parts. Another thing was trying to constantly remember and format commit messages to link back to the Kanban board (it was a hassle). Moving forward, I want to make sure I map out all future feature requirements fully before writing the code, while also doing tasks earlier in the timeline to avoid being overwhemled with multiple things.
- Camila: I think the testing in this iteration was good enough. We did get branch coverage and line coverage. I think a way to improve the process for future iteration is to work on it earlier, set different

times to do the code, and communicate better with teammates. I know that I got quite confused with front end aspect which could have been clearer.

- Alejandro: This iteration was pretty intense because I has to implement poker gameplay logic, handling the deck utility and community cards (flop, turn, river) to hand evaluation for selecting the winner and turn-based system. The biggest problem for me was dealing with architectural mismatches. My core logic and state management approach didn't match up with how other parts of the system were implemented, so I had to spend a massive chunk of time completely refactoring `activeTable.js`, `tableController.js`, and the frontend views just to bridge the gap and make everything actually line up. Moving forward, I learned that we need much tighter alignment and communication on our backend assumptions and interface designs *before* we start coding, which will save us from having to do stressful structural rewrites mid-sprint.

- Gigi: The challenges of this iteration was trying to solve the merge conflicts that were happening due to each team member committing and pushing at the same time. Additionally, there were confusion on understanding each other's code when there were dependencies. To address these problems, I believe we can designate a specific time interval that each team member can work on the code with respect to the dependencies in the user stories that each are working on. According to plan, every team member was able to do complete their user stories, but we took a lot of time to solve merges and conflicts with our respective parts of the code. Again, if we have different deadlines for each team member to complete their part, we can progress without dependencies and hopefully no merge conflicts.

5.3 Iteration/Sprint 2

5.3.1 Planning

For this iteration, our main focus was on expanding the gameplay mechanics, getting account management and guest technicalities sorted out, and adding some cool community features. We planned around 27 story points total for this sprint to bridge the gap between backend poker logic and a fully interactive web dashboard.

To make the poker matches work holistically , we spent a big chunk of our stories diving into core gameplay actions and edge cases. This meant building out winner hand calculations for showdowns (PG15), adding muck capabilities so players can show or hide cards (PG16), and handling folded player tracking (PG17). We also worked on blocker checks and calls (PG18), improved countdown timers (PG16), and fine-tuned extended turn durations (PE23) to keep betting rounds flowing smoothly.

Alongside the active tables, we wanted to expand the users' out-of-game actions. That included being able to create new customized tables (PE6) and being able to sort all tables (PE22) so players can easily browse active rooms (and break them down by categories). We also wanted to set up the match history suite (PE19) so admin/users can review past games and reconstruct board states (admin has access to all games while players can only see their own games). Account security and platform moderation were huge priorities for us too. We kept building out our non-game management features, specifically creating the player reporting system (A2) where users can flag infractions and admins can issue warnings that the offending party can see, alongside password reset tools (PE21) so that users can reset passwords while still keeping their account secure

To balance accessibility with data integrity, we also worked on guest account logic (G1), defining what guests are allowed to do while making sure points are reliably updated for registered users after gameplay (G5) without unauthenticated guest accounts messing up persistent state records. On the social side, we added in-game chat emotes (PE23) so people can react during hands and a spectating mode (PE9) for watching active tables live without playing. Finally, to clean up our code before merging everything into main, we want to do a thorough refactoring sweep (D1) to scrub redundant files, keep our logic strictly in controllers, and clean up any repeating code patterns hanging around.

5.3.2 Work Done

****EVERYBODY DO THIS SECTION**** [Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

- Camila:
  1. PE6 (Create Tables) — (12 hrs) Complete/Incomplete: Completed? I used item.html, tableManager, activeTable.js, tableController.js, and app.js, index.html, lobby.js, lobby.html, tableDisplay.js, and tableDisplay.htlm to work through to create mutiple table instances, new functions for the tables, connecting front end to back end, editing the html so that it would dyamically update to the specific table rather than the three hard coded tables.
  2. PE13 (Lobby) — (3 hrs) Complete/Incomplete: Completed? The main two files need to complete this was lobby.html, and lobby.js. These two were the main files to create the lobby itself. Then in item.html, tableManager, activeTable.js, tableController.js, and app.js were necessary for creating the multiple table logic on the backend to support the creation of new tables for the ranked game. I need to modify both the front and back end to make the table dynamic despite it not being visible.
- Alejandro:
  1. PG15 (Winner Hand) — (5 hrs) Complete: Implemented automated showdown hand collection. Built the synchronized client-side sequenced reveal system (SHOWDOWN_REVEAL via Socket.io and tableDisplay.js) to animate active hole card reveals across all relevant players simultaneously.
  2. PG16 (Muck) — (6 hrs) Complete: Added folding and mucking logic to the player manager and active table session (playerManager.js, activeTable.js), ensuring folded and mucking players' cards and are excluded from showdown equity distribution while preserving valid non-folded player cards for review.
  3. PG18 (Block check/call) — (2 hrs) Complete: Implemented strict turn validation and state gating in tableController.js and tableDisplay.js. Enforced rules to block check buttons when facing an active bet, validate call amounts against available chips, and disable action buttons dynamically when it is not the player's turn or when the player has already acted.
  4. D1 (Refactor) — (4 hrs) Complete: Cleaned up code architecture across backend controllers and models, removing redundant state wiping during showdown sequences, resolving race conditions between HTTP actions and Socket.io broadcasts, and improving overall maintainability and error-handling consistency.
  5. PE19 (Match History) — (6 hrs) Complete: Designed and implemented the persistence architecture for completed matches, including the GameDao Mongoose schema to store session metadata, initial seating orders, positional indices (dealer, small blind, big blind), and shuffle seeds. Created the /api/games endpoints and server-side reconstruction logic to evaluate player hand scores against common board cards.
  6. A6 (History Screen) — (5 hrs) Complete: Developed the game review and reconstruction dashboard frontend (reviewGame.html and reviewGame.js), featuring a session summary panel, dynamic board breakdowns across betting streets, and structured player seating tables displaying hole cards and evaluation scores alongside chronological action logs.
- Gigi:
  1. G1 (Guest Account) — (6 hrs) Complete: Separated user and guest account interfaces cleanly across the frontend and backend, enabling guest users to browse lobbies and join active tables independently with isolated session state.
  2. G5 (Keep Points) — (3 hrs) Incomplete: Made limited progress on points persistence for guest sessions, as implementation was directly blocked pending the completion and stabilization of the guest account architecture.

3. PE23 (Emote) — (0 hrs) Incomplete: Did not implement within this development cycle due to prior task prioritizing and scope allocation toward core game engine state stability.
4. PE23 (Message Visibility) — (4 hrs) Incomplete: Achieved basic messaging functionality across table channels, but full chat visibility controls, filtering rules, and edge-case rendering remain unfinished. Spent time debugging the implementation of a second message box.
5. PE17 (Folded Players) — (4 hrs) Complete: Implemented folded player state handling and UI card graying out within `activeTable.js` and `tableDisplay.js`, accurately excluding folded hands from pot eligibility while retaining board visibility.
6. PG16 (Better Timers) — (6 hrs) Complete: Built a resilient Socket.io timer spike and turn ticker in `tableController.js` and `server.js` that handles turn countdowns, triggers auto-folds on timeout, and gracefully manages player disconnects/reconnects.
7. PE9 Spectating: (4 hrs) Incomplete: Explored initial spectator views and real-time state listeners, but encountered active synchronization bugs and seat tracking issues, particularly following the multi-table refactor.

- Bonita:

1. A2 (Report) — (9-10 hrs) Complete: For this story, I started by making the backend data layer and controller logic, creating 'ReportDao' to manage a new database and 'ReportController' to handle report submissions, retrieval, resolution updates, and the administrative warning pipeline. I also had to update 'UserController' to properly handle incoming user warnings and notification messages sent by admins. On the frontend, I designed and integrated the user and admin dashboard interfaces in HTML. This included building out a reporting modal for players to report infractions, an admin review panel to inspect pending tickets and issue warnings with custom messages, and a player notification inbox. Offending Players can click headers to expand message previews, and securely delete notifications (this message tool can probably be used later on to send messages for a variety of things not only admin warnings). I also added Jest unit test suite covering both the controller and DAO layers to verify success paths, error handling, and edge cases.
2. PE21 (Reset Password) — (0 hrs) Incomplete: Although I had planned on working on this task, many of my teammates needed to work on the same files that I would have had to work on the complete this task. I had already met my point requirements with A2 and my teammates had not met their point requirments so I decided to let them work on the files and stash this task for the next iteration. I wanted to be considerate of them because last time I was working on files that they needed to work on (when I already had points that I needed) which caused issues for them.

5.3.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

Although our overall coverage is relatively high, with 94.51% statement coverage and 95.18% line coverage, I don't think it is fully sufficient yet because we still have 2 failing test suites and 7 failing tests. Our branch coverage is also lower at 82.78%, meaning some conditional paths and edge cases are not being tested. Certain files, such as `tableController.js`, `DbConnection.js`, and `ReportDao.js`, have lower coverage because their error-handling logic and more complex conditions are harder to test with standard unit tests.

To improve our coverage, I think we should first fix the failing tests to establish a stable baseline. After that, we can focus on testing the uncovered sections of `tableController.js`, mock database connection failures in `DbConnection.js`, and add tests for the remaining DAO error-handling cases. This would help us improve not only our coverage percentages but also the reliability of the application.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	94.51	82.78	97.96	95.18	
controller	88.97	72.6	94.87	89.39	
ReportController.js	88.46	88.46	100	88.46	58-59,92-93,152-153
UserController.js	95.62	74.44	95	97.1	23-24,218,408,454
tableController.js	82.08	69.31	93.33	81.87	109,159-165,245-303,385,427,458,498
model	96.26	88.47	98.11	97.56	
DbConnection.js	81.25	50	100	85.71	15-16
GameDao.js	100	100	100	100	
ReportDao.js	80	100	66.66	80	45-46
UserDao.js	93.93	66.66	100	93.93	116-117
activeTable.js	96.91	87.68	98.5	97.94	28,107,247,260,325,347,376,437,446
playerManager.js	95.71	85	100	100	22-37,42,82,90,109
tableManager.js	100	100	100	100	
util	100	96.72	100	100	
PasswordUtil.js	100	100	100	100	
pokerLogic.js	100	96.72	100	100	15,260

Test Suites: 2 failed, 10 passed, 12 total
Tests: 7 failed, 220 passed, 227 total
Snapshots: 0 total
Time: 3.641 s
Ran all test suites.

5.3.4 Retrospective & Reflection

****EVERYBODY DO THIS SECTION**** [What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

- Camila: This iteration was more challenging than I originally anticipated. The tasks in plain english seemed simple. Boy was I wrong. My main focus was implementing the Lobby and Create Table features, which required significant backend refactoring to support multiple independent tables and updating the frontend to dynamically display information. The problem was that we had encoded our logic to support ONE table. One of the biggest challenges was ensuring these changes didn't interfere with existing functionality, as fixing one issue introduced another. Then when I returned, to test the newly implemented code, I'd see issues that had to relate to multiple tables. I also realized that making too many changes at once made debugging unnecessarily difficult. I need to commit more. Moving forward, I plan to implement smaller changes and test each feature more thoroughly before continuing. I find myself thinking that fixing one main issue and committing, which usually takes multiple files is not the best idea. Overall, this iteration taught me the importance of scalability, proper testing, and maintaining consistency between the frontend and backend, although it definitely cost me more time and patience than expected. Also all of my tasks were heavily reliant on each other to complete each other. If one wasn't fully complete then the others could not be fully complete.

- Alejandro: This iteration presented unique technical challenges, particularly around asynchronous state management, frontend-backend synchronization, and deterministic game state reconstruction. One major hurdle we encountered was ensuring that the automated showdown sequence and card reveal animations (SHOWDOWN_REVEAL) synchronized correctly across all connected clients via Socket.io without being prematurely wiped out by asynchronous UI refreshes or rapid server-side session resets. To address this, we decoupled the endgame lobby resets from the immediate post-game evaluation, allowing hole cards and winner highlights to persist securely for all participants until the next game session was explicitly initiated. Additionally, coordinating match history persistence (GameDao) with deterministic PRNG seeding required meticulous care to ensure hands could be accurately reconstructed server-side. Moving forward, I plan to write more modular unit tests for real-time WebSocket events and establish stricter separation between UI rendering cycles and state

mutations. Technically, this iteration taught me a great deal about the nuances of real-time state synchronization, handling race conditions between HTTP REST endpoints and Socket.io broadcasts, and managing DOM persistence during animated component sequences.

- Gigi: During this iteration, one of my biggest challenges was making sure different parts of the poker game worked together correctly. I ran into issues with user and guest IDs displaying incorrectly at the table, timer functionality, and player actions. Some of these problems required checking multiple files because the issue was not always in the feature I was working on. Testing was also challenging because some functions did not behave as expected in certain situations, so I had to review the existing code and add or modify Jest tests to cover more cases. As a result, I spent more time fixing existing functionality than I originally planned, which delayed progress on some of my assigned features.

- For the next iteration, I think we can improve by testing features more frequently and communicating more with teammates about changes to shared files and features that depend on each other. This could help us catch integration issues earlier instead of dealing with several problems at once. Personally, I learned how important it is to understand how different parts of the backend interact before making changes. I also became more comfortable using Jest to test different scenarios and learned that even small changes to user sessions, player information, or game timers can affect the rest of the application. Overall, this iteration taught me that completing a feature means making sure it works with the entire system, not just on its own.

- Bonita: Honestly this iteration went very smoothly for me. I my part much earlier than the due date so I was able to add files without running into any merging conflicts since all my other teammates had not started working yet. In terms of the actual work I think the main issue I had was that I had to change the user schema once again (to add a warnings section for users) which caused issues with older accounts, ultimately I had to delete and create new accounts to ensure everything was working correctly. Besides that I feel like adding all the files for report was very straight forward. Honestly I had a lot of time left over and I felt like I could have done more but I am holding myself back because last iteration I wanted to add more stories and I ended up messing up my other teammates stories (caused merge conflicts for them) which in turn caused my stuff to mess up later on as well.

5.4 Iteration/Sprint 3

5.4.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.4.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.4.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.4.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.5 Iteration/Sprint 4

[CS496 has 5 sprints. CS482 only has only 3 sprints (remove Iterations 4 and 5 from this doc if you are writing a doc for 482]

5.5.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.5.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.5.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.5.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.6 Iteration/Sprint 5

5.6.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.6.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.6.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.6.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

6 Final Remarks

6.1 Overall Progress

[Have you completed everything? If so, present evidence on how you brought value to your client, and the overall client satisfaction. Otherwise, estimate how much progress you done and how long it would take to finish this project. Be concrete about your progress, you know how many story points your software is, how many points you completed (this shows your progress). You also how many points your team delivers at each iteration, therefore you can estimate how many more iterations it would take to finish the leftover points (show the math).]

6.2 Project Reflection

[Your personal reflection on the project. What lessons did you learned. What would you have done differently? How can you do better work in future projects? You may write this as a team or per person (or both — if all your iterations were team reflections, then it would be better to write individual reflections here)]

Appendix

[Appendix section if needed]